

You are a student working on a deep learning coding problem set as per the attached file.

Can you solve problem 4 as noted in the PDF. The instructions are as follows:

Implement the reparametrization trick in `sample_gaussian` in `utils.py`.

Implement `negative_elbo_bound` in `models/vae.py`

The files `utils.py` and `vae.py` are attached as well. Please share your implementations!

Grok demonstrated production-level coding habits. Specifically, its use of torch functions instead of manual shape passing ensures the code is device-agnostic. It also correctly adhered to the specific variable definitions

Implementation for `sample_gaussian` in `utils.py`

The reparameterization trick allows sampling from a Gaussian $\mathcal{N}(\mu, \sigma^2)$ in a differentiable way by sampling $\epsilon \sim \mathcal{N}(0, 1)$ and computing $z = \mu + \sigma \cdot \epsilon$, where $\sigma = \sqrt{v}$ (since `v` is the variance).

Here's the completed function:

This implementation is perfect

Python

```
def sample_gaussian(m, v):
    """
    Element-wise application reparameterization trick to sample from Gaussian

    Args:
        m: tensor: (batch, ...): Mean
        v: tensor: (batch, ...): Variance

    Return:
        z: tensor: (batch, ...): Samples
    """
    eps = torch.randn_like(v)
    z = m + eps * torch.sqrt(v)
    return z
```

Implementation for `negative_elbo_bound` in `models/vae.py`

The negative ELBO is the sum of the KL divergence term (between approximate posterior $q(z|x)$ and prior $p(z)$) and the reconstruction loss term (negative expected log-likelihood under the decoder, here using Bernoulli for binary data like binarized MNIST). We compute means over the batch to get scalars.

Here's the completed method:

Python

```
def negative_elbo_bound(self, x):
    """
    Computes the Evidence Lower Bound, KL and, Reconstruction costs

    Args:
        x: tensor: (batch, dim): Observations

    Returns:
        nelbo: tensor: (): Negative evidence lower bound
        kl: tensor: (): ELBO KL divergence to prior
        rec: tensor: (): ELBO Reconstruction term
    """
    mean, var = self.enc.encode(x)
    z = ut.sample_gaussian(mean, var)
    logits = self.dec.decode(z)

    kl = ut.kl_normal(mean, var, self.z_prior[0], self.z_prior[1])
    rec = -ut.log_bernoulli_with_logits(x, logits)
    nelbo = kl + rec

    nelbo = nelbo.mean()
    kl = kl.mean()
    rec = rec.mean()

    return nelbo, kl, rec
```

This implementation was also perfect!

Can you tell me how to visualize samples from my VAE (Visualize 200 digits (generate a single image tiled in a grid of 10×20 digits) sampled from $p_\theta(x)$)?

To visualize 200 digits sampled from $p_\theta(x)$ in your VAE, arranged in a single tiled image grid of 10 rows by 20 columns, add and run the following code after training (e.g., in your experiment script or a separate visualization script). This assumes your trained model has been saved via `save_model_by_name` (as seen in `utils.py`) at global step 10000, and that you're using the default VAE configuration from `vae.py` (e.g., `z_dim=2`). Adjust the `z_dim` or other init params if your setup differs. The samples are generated via Bernoulli sampling for binary digits, which matches the model's `p_\theta(x|z)`.

Python

```
import torch
import matplotlib.pyplot as plt
import numpy as np
from models.vae import VAE
import utils as ut

# Load the trained model (adjust nn, z_dim, name, and global_step as needed)
model = VAE(nn='v1', name='vae', z_dim=2)
ut.load_model_by_name(model, global_step=10000)

# Set to evaluation mode
model.eval()

# Generate 200 samples from  $p_\theta(x) = \int p_\theta(x|z) p(z) dz$ 
# This samples  $z$  from prior, then  $x \sim \text{Bernoulli}(p_\theta(x|z))$ 
with torch.no_grad():
    samples = model.sample_x(200).cpu().numpy().reshape(200, 28, 28)

# Create a 10x20 grid (280x560 pixels)
grid = np.zeros((10 * 28, 20 * 28))
for i in range(10):
    for j in range(20):
        grid[i * 28 : (i + 1) * 28, j * 28 : (j + 1) * 28] = samples[i * 20 + j]

# Visualize and save/show
plt.figure(figsize=(20, 10))
plt.imshow(grid, cmap='gray')
plt.axis('off')
plt.savefig('vae_samples_10x20.png') # Save to file
# plt.show() # Uncomment to display interactively
```

Run this code to generate and save the tiled image as 'vae_samples_10x20.png'. If you prefer grayscale probabilities instead of binary samples (e.g., for smoother visuals), replace `model.sample_x(200)` with `model.sample_sigmoid(200)`.